Q4: Complete Request Data Flow Trace

> ***Question:*** *Trace the exact path of a request through the most important POST endpoint — from router, through Pydantic validation, into the service layer, and finally the SQLAlchemy commit.*

Chosen Endpoint: `POST /api/v1/hardware/camera`

This is the single most consequential endpoint in the system. One request can:

- Update a trip's crowding state
- Create two database records (CameraReading + CrowdingEvent)
- Spawn a new Trip record (auto-dispatch)
- Lock database rows to prevent race conditions
- Broadcast a real-time WebSocket alert to managers

It exercises every layer of the stack: middleware, API key auth, Pydantic, ORM, service business logic, row-level locking, multiple commits, and WebSocket push.

The Full Flow at a Glance

```
ESP32-CAM Hardware
       |
       | POST /api/v1/hardware/camera
       | Headers: X-Hardware-API-Key: <key>
       | Body:    {"trip_id": 42, "passenger_count": 46}
       v
[1] SanitizationMiddleware       — sanitize JSON strings (bleach)
[2] IdempotencyMiddleware         — check Idempotency-Key (if present)
[3] CORSMiddleware                — skip (not a browser request)
[4] slowapi RateLimiter           — enforce 60 req/min per IP
[5] Router match                  — hardware.router, POST /camera
[6] verify_hardware_api_key()     — constant-time API key check
[7] Pydantic: CameraData          — validate trip_id (int), passenger_count (int)
[8] get_db()                      — open AsyncSession
[9] Endpoint handler              — ingest_camera()
[10] DB queries                   — fetch Trip, fetch Vehicle
[11] Business logic               — calculate crowding_score
[12] trigger_auto_dispatch()      — if score >= 0.90
      [12a] SELECT Driver WITH FOR UPDATE
      [12b] SELECT Vehicle WITH FOR UPDATE
      [12c] INSERT Trip (extra dispatch)
      [12d] INSERT DriverExchange
      [12e] db.commit()
[13] INSERT CameraReading         — persist raw sensor data
[14] INSERT CrowdingEvent         — if score >= 0.70
[15] db.commit()                  — finalize all remaining writes
[16] WebSocket broadcast          — push crowding_alert to MANAGER role
[17] Return JSON response
```

Step 1-4: Middleware Pipeline

SanitizationMiddleware

The body `{"trip_id": 42, "passenger_count": 46}` passes through `bleach.clean()` on all string values. Since both fields are integers, no mutation occurs. The body is passed through unchanged.

IdempotencyMiddleware

No `Idempotency-Key` header is sent by hardware devices, so this middleware calls `call_next(request)` immediately and passes the request through.

CORSMiddleware

Hardware devices do not send an `Origin` header. The middleware takes no action.

Rate Limiter (slowapi)

The `@limiter.limit("60/minute")` decorator on this specific endpoint overrides the global 100/min default. The limiter reads `request.client.host` (the device IP) and checks the in-memory counter. If under 60, the request proceeds.

```
@router.post("/camera", status_code=status.HTTP_200_OK)
@limiter.limit("60/minute")
async def ingest_camera(request: Request, data: CameraData, ...):
```

Step 5-6: Router Match and API Key Verification

FastAPI matches `POST /api/v1/hardware/camera` to `hardware.router` . Before calling the handler, FastAPI resolves the router-level dependency:

```
router = APIRouter(dependencies=[Depends(verify_hardware_api_key)])
```

The dependency function runs:

```
async def verify_hardware_api_key(x_hardware_api_key: Optional[str] = Header(None)):
    key = x_hardware_api_key or ""
    if not secrets.compare_digest(key, settings.HARDWARE_API_KEY):
        raise HTTPException(403, "Invalid or missing hardware API key")
```

`secrets.compare_digest` compares both strings in constant time, regardless of length, to prevent timing attacks. If the header is missing or wrong, a `403 Forbidden` is raised here and the handler never runs.

Step 7: Pydantic Validation — `CameraData`

FastAPI deserializes the raw JSON body into the `CameraData` schema:

```
class CameraData(BaseModel):
    trip_id: int
    passenger_count: int
```

This schema inherits from plain `BaseModel` (not `BaseSchema`), because it is inbound-only — it never needs to serialize from an ORM object.

What Pydantic does here:

- Coerces `"trip_id": 42` to Python `int`
- Coerces `"passenger_count": 46` to Python `int`
- Rejects non-integer values with `HTTP 422 Unprocessable Entity`
- Rejects missing fields with `HTTP 422`
- The global `validation_exception_handler` in `main.py` formats the 422 response

If validation passes, `data` is a fully typed Python object ready for use:

```
data.trip_id         # → 42  (int)
data.passenger_count # → 46  (int)
```

Step 8: Database Session Injection

The `get_db()` dependency opens an `AsyncSession`:

```
async def get_db():
    async with AsyncSessionLocal() as session:
        try:
            yield session
        finally:
            await session.close()
```

The session is created from `async_sessionmaker` bound to the shared async engine. It is configured with `expire_on_commit=False` and `autoflush=False` — meaning objects do not expire after commit (safe for async) and SQL is not automatically flushed before queries.

The session is injected into the handler as `db: AsyncSession`.

Step 9-11: Endpoint Handler — Business Logic

```
async def ingest_camera(request: Request, data: CameraData, db: AsyncSession):
```

9a. Fetch Trip

```
trip = await db.scalar(select(Trip).where(Trip.id == data.trip_id))
if not trip:
    raise HTTPException(status_code=404, detail="Trip not found")
```

SQLAlchemy emits:

```
SELECT trips.* FROM trips WHERE trips.id = 42 LIMIT 1
```

If no trip is found, the handler raises `HTTP 404` and the session is closed by `get_db()` .

9b. Fetch Vehicle

```
vehicle = await db.scalar(select(Vehicle).where(Vehicle.id == trip.vehicle_id))
if not vehicle:
    raise HTTPException(status_code=404, detail="Vehicle not found for trip")
```

SQLAlchemy emits:

```
SELECT vehicles.* FROM vehicles WHERE vehicles.id = <trip.vehicle_id> LIMIT 1
```

9c. Calculate Crowding Score

```
capacity = vehicle.capacity  # e.g. 50
trip.passenger_count = data.passenger_count  # 46

crowding_score = min(data.passenger_count / capacity, 1.0)
# → min(46 / 50, 1.0) = min(0.92, 1.0) = 0.92

trip.crowding_score = 0.92
trip.is_crowded = trip.crowding_score > 0.90  # → True
```

This mutation marks the `Trip` ORM object as **dirty** in the SQLAlchemy identity map. No SQL is emitted yet — changes are tracked in-memory.

Edge case guard: If `vehicle.capacity <= 0` , the crowding score is set to `-1.0` and the `is_crowded` flag stays False, preventing a `ZeroDivisionError` .

9d. INSERT CameraReading

```
camera_reading = CameraReading(
    trip_id=trip.id,          # 42
    vehicle_id=vehicle.id,
    passenger_count=46,
    crowding_score=0.92,
)
db.add(camera_reading)
```

The `CameraReading` object is added to the session's pending INSERT queue. Still no SQL emitted.

Step 12: Auto-Dispatch (crowding_score >= 0.90)

Since `trip.is_crowded = True` , the handler calls:

```
auto_dispatched = await trigger_auto_dispatch(trip.id, db)
```

This is the deepest part of the flow. The same `db` session is passed — both the handler and the service share one transaction context.

12a. Lock DRIVER_3 with `FOR UPDATE`

```
driver = await db.scalar(
    select(Driver)
    .join(RotationAssignment, Driver.id == RotationAssignment.driver_id)
    .where(
        Driver.status == DriverStatus.ACTIVE,
        RotationAssignment.shift_date == today,
        RotationAssignment.position == RotationPosition.DRIVER_3
    )
    .limit(1).with_for_update()
)
```

SQLAlchemy emits:

```
SELECT drivers.*
FROM drivers
JOIN rotation_assignments ON drivers.id = rotation_assignments.driver_id
WHERE drivers.status = 'ACTIVE'
  AND rotation_assignments.shift_date = '2026-04-16'
  AND rotation_assignments.position = 'DRIVER_3'
LIMIT 1
FOR UPDATE
```

`FOR UPDATE` places a **row-level exclusive lock** on the matched `Driver` row. Any other concurrent transaction trying to select or update this row will block until this transaction commits or rolls back. This prevents two simultaneous crowding events from double-dispatching the same driver.

If no DRIVER_3 is available, `trigger_auto_dispatch` returns `False` immediately.

12b. Lock Free Vehicle with `FOR UPDATE`

```
vehicle = await db.scalar(
    select(Vehicle).where(Vehicle.status == VehicleStatus.FREE).limit(1).with_for_update()
)
```

Same row-level lock applied to the vehicle row. Prevents two dispatches from claiming the same bus.

12c. Build the Extra Dispatch Trip

```
now_utc = datetime.now(timezone.utc)
dispatch_start = now_utc + timedelta(minutes=5)

new_trip = Trip(
    driver_id=driver.id,
    vehicle_id=vehicle.id,
    route_id=trip.route_id,
    direction=trip.direction,
    status=TripStatus.SCHEDULED,
    trip_number=f"EXT-RT{trip.route_id}-{now_utc.strftime('%H%M')}-{uuid4().hex[:6]}",
    scheduled_start=dispatch_start,
    scheduled_end=dispatch_start + timedelta(minutes=route.estimated_time_minutes),
    is_extra_dispatch=True,
    notes=f"Auto-dispatched to relieve overload on {trip.trip_number}"
)
db.add(new_trip)

driver.status = DriverStatus.ON_TRIP
```

```
vehicle.status = VehicleStatus.ASSIGNED
```

12d. Flush to Get new_trip.id

```
await db.flush()
```

`flush()` sends all pending SQL to the database **within the current transaction** but does NOT commit. This is necessary because the next step (`DriverExchange`) needs `new_trip.id` as a foreign key, which is only assigned after the INSERT is sent to PostgreSQL (auto-increment).

SQLAlchemy emits:

```
INSERT INTO trips (driver_id, vehicle_id, route_id, direction, status, trip_number, ...)
VALUES (...) RETURNING trips.id
```

The returned `id` is populated on `new_trip.id` .

12e. INSERT DriverExchange

```
exchange = DriverExchange(
    rotation_assignment_id=outgoing_assignment_id,
    outgoing_driver_id=trip.driver_id,
    incoming_driver_id=driver.id,
    reason=ReplacementReason.EMERGENCY_CROWDING,
    exchange_time=datetime.now(timezone.utc),
    trip_id=new_trip.id,   # available because of the flush above
    notes=f"Auto-dispatch for crowding on trip {trip.trip_number}",
)
db.add(exchange)
```

12f. First Commit (inside trigger_auto_dispatch)

```
await db.commit()
```

This commits the sub-transaction that was built inside `trigger_auto_dispatch` . At this point, PostgreSQL durably writes:

- 1 new `Trip` row (the extra dispatch)
- 1 new `DriverExchange` row
- Updated `Driver.status` = `ON_TRIP`
- Updated `Vehicle.status` = `ASSIGNED`
- The `FOR UPDATE` locks are released

Returns `True` to the calling handler.

Step 13-14: Persist CameraReading and CrowdingEvent

Back in the handler, after `trigger_auto_dispatch` returns:

```
# camera_reading was already added in step 9d — still pending in session
# Now add CrowdingEvent since score >= 0.70

if trip.crowding_score >= 0.70:
```

```
    crowding_event = CrowdingEvent(
        trip_id=trip.id,
        vehicle_id=vehicle.id,
        crowding_score=0.92,
        passenger_count=46,
        auto_dispatch_triggered=True,  # set from trigger_auto_dispatch return value
    )
    db.add(crowding_event)
```

Step 15: Final Commit

```
await db.commit()
```

This final commit flushes and persists everything still pending in the session:

- `UPDATE trips SET passenger_count=46, crowding_score=0.92, is_crowded=True WHERE id=42`
- `INSERT INTO camera_readings (trip_id, vehicle_id, passenger_count, crowding_score, ...) VALUES (...)`
- `INSERT INTO crowding_events (trip_id, vehicle_id, crowding_score, passenger_count, auto_dispatch_triggered, ...) VALUES (...)`

After commit, the session's identity map is cleared (though `expire_on_commit=False` means loaded objects retain their attribute values in memory).

Total database writes in this one request (crowding >= 90%):

Table	Operation	Trigger
`trips` (original)	UPDATE	crowding score + is_crowded flag
`trips` (extra dispatch)	INSERT	auto-dispatch
`driver_exchanges`	INSERT	auto-dispatch
`drivers`	UPDATE	status ON_TRIP
`vehicles`	UPDATE	status ASSIGNED
`camera_readings`	INSERT	raw sensor data
`crowding_events`	INSERT	score >= 70%

Step 16: WebSocket Broadcast

After the commit, the handler pushes a real-time alert to all connected Manager clients:

```
if trip.is_crowded:
    alert_payload = {
        "type": "crowding_alert",
        "severity": "HIGH",
```

```
            "trip_id": trip.id,
            "route_id": trip.route_id,
            "crowding_score": 0.92,
            "message": "CRITICAL CROWDING on Trip TRP-3-0-O. Auto-dispatch successful."
        }
        await manager.broadcast_to_role("MANAGER", alert_payload)
```

`broadcast_to_role("MANAGER", ...)` iterates the `active_connections["MANAGER"]` set in `ConnectionManager` and calls `websocket.send_json(alert_payload)` on each connected socket. Failed connections are silently removed from the set.

This all happens **after** the database commit — the data is durable before the notification is sent.

Step 17: HTTP Response

The handler returns:

```
return {"message": "Crowding updated", "score": 0.92}
```

FastAPI serializes this dict to JSON. The response is:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"message": "Crowding updated", "score": 0.92}
```

The `get_db()` generator's `finally` block runs, calling `await session.close()` , which returns the connection to the pool.

Complete SQL Timeline

In order of execution during a single request (crowding >= 90% scenario):

```
-- Step 9a: Fetch trip
SELECT * FROM trips WHERE id = 42 LIMIT 1;

-- Step 9b: Fetch vehicle
SELECT * FROM vehicles WHERE id = <trip.vehicle_id> LIMIT 1;

-- Step 12a: Lock DRIVER_3
SELECT drivers.* FROM drivers
JOIN rotation_assignments ON ...
WHERE ... position = 'DRIVER_3'
LIMIT 1 FOR UPDATE;

-- Step 12b: Lock free vehicle
SELECT * FROM vehicles WHERE status = 'FREE' LIMIT 1 FOR UPDATE;

-- Step 12c: Fetch route for trip duration
SELECT * FROM routes WHERE id = <trip.route_id> LIMIT 1;

-- Step 12d: Resolve assignment FK
```

```
SELECT * FROM rotation_assignments WHERE ... LIMIT 1;

-- Step 12e: flush() — INSERT new trip
INSERT INTO trips (...) VALUES (...) RETURNING id;

-- First commit (trigger_auto_dispatch)
UPDATE drivers SET status = 'ON_TRIP' WHERE id = <driver.id>;
UPDATE vehicles SET status = 'ASSIGNED' WHERE id = <vehicle.id>;
INSERT INTO driver_exchanges (...) VALUES (...);
COMMIT;

-- Step 15: Final commit
UPDATE trips SET passenger_count=46, crowding_score=0.92, is_crowded=true WHERE id=42;
INSERT INTO camera_readings (...) VALUES (...);
INSERT INTO crowding_events (...) VALUES (...);
COMMIT;
```

What Happens When crowding_score Is Between 0.70 and 0.90

The auto-dispatch branch is skipped. Only these writes occur:

```
SELECT trips WHERE id = 42 LIMIT 1;
SELECT vehicles WHERE id = ... LIMIT 1;
UPDATE trips SET ... WHERE id = 42;
INSERT INTO camera_readings (...);
INSERT INTO crowding_events (...);   ← because >= 70%
COMMIT;
```

What Happens When crowding_score Is Below 0.70

No auto-dispatch, no CrowdingEvent:

```
SELECT trips WHERE id = 42 LIMIT 1;
SELECT vehicles WHERE id = ... LIMIT 1;
UPDATE trips SET ... WHERE id = 42;
INSERT INTO camera_readings (...);
COMMIT;
```

Error Paths and Guarantees

Error	Where Caught	Response	Side Effects
Missing/wrong API key	`verify_hardware_api_key`	403	Nothing written
Invalid JSON / wrong types	Pydantic	422	Nothing written

Error	Where Caught	Response	Side Effects
Trip not found	Endpoint	404	Nothing written
Vehicle not found	Endpoint	404	Nothing written
No DRIVER_3 available	`trigger_auto_dispatch`	200 (success=False)	No dispatch, rest continues
No free vehicle available	`trigger_auto_dispatch`	200 (success=False)	No dispatch, rest continues
Rate limit exceeded	slowapi	429	Nothing written
Unhandled exception	Global handler in main.py	500	Session rolled back by `get_db()` finally

---

Generated: 2026-04-16